

Startup Matchmaking: Automation Approach

Task 2 write-up: approach and architecture

Today, matching an incoming request to suitable startups means opening a 600+ row Excel file and manually checking it against whatever the requester asked for. That takes about 7 hours a week across 10+ requests. The goal here is to turn that process, read the email, search the sheet, write a reply, into a short pipeline with one required human checkpoint before anything goes out. We already run and fully control a Hetzner server for the startin.lv website, so this tool is designed to run there as its own self-hosted service, rather than adding a stack of new paid subscriptions.

Architecture and data flow

1. Intake	Requests arrive at the team's existing mailbox. A small script checks it every few minutes and passes new messages to the matchmaking app, so there is no separate inbound-email service to pay for. Duplicate emails are ignored automatically.
2. Understand the ask	Claude reads the free-text email and pulls out structured criteria: sector, stage, geography, ticket size, and deadline. It also separates must-have requirements from nice-to-haves. If a request is unclear, the system drafts a clarifying question instead of guessing.
3. Match	Startups that fail a must-have criterion are excluded. The rest are ranked by a rules-based scoring system, the same approach used in the Task 1 prototype submitted alongside this document, so the reasoning behind every rank stays visible instead of hidden inside a model.
4. Shortlist draft	The top 15-20 startups are shown with a one-line note explaining why each one fits, plus a link back to the exact criteria it matched. This lands in a review screen inside the app.
5. Human review	A team member checks the list, edits it if needed, and approves it. Nothing is sent to a requester without this step.
6. Send and log	The approved shortlist goes out through the team's existing mailbox. A bounce reopens the request. Every request, shortlist, and edit is logged for future reference.

The guiding principle throughout is that a deterministic layer, not an AI model, always decides who is eligible and how they rank. AI is used only where it genuinely helps: reading free text and writing short explanations. It never makes the final decision to send something out, and any text pulled from emails or websites is treated purely as information, never as an instruction for the system to follow. The matchmaking tool is built as its own app, separate from the website's codebase. Running two apps side by side needs more memory than the current server has spare, since even a single build already needs extra swap space, so this plan assumes upgrading the server to 4 vCPUs and 8GB of RAM before both run together.

Tools and estimated running cost

Item	Tool	Est. monthly cost
Hosting (both apps)	Hetzner VPS, upgraded to 4 vCPU / 8GB so the website and the matchmaking app can run as two independent services without competing for memory during builds	About €7.70 (up from about €4.35 today)
Startup and request database	Postgres on the same VPS, in its own database	€0 new
Inbound email	Existing team mailbox, checked by a small script	€0 new
Outbound email	Existing email account already used by the site	€0 new
Matching logic (filters and scoring)	Runs inside the matchmaking app, no external service	€0
AI (reading requests, writing rationale)	Claude API, pay-per-use, roughly 50 requests a month	€10 to €25
Total new cost		roughly €13 to €28 a month

That is a small fraction of the roughly 28 hours a month this process currently costs the team, while keeping everything on infrastructure the team already owns and understands.

Verifying match quality, and keeping a human in the loop

- A person always reviews the shortlist, and can edit it, before anything is sent. The system never emails a requester on its own.
- Before launch, we would test the system against about 20 real past requests, including tricky or no-match cases. Half would be used to tune the scoring; the other half would be kept aside purely to check accuracy. We would track what share of suggested startups are actually suitable, and treat any case where a must-have requirement was ignored as a bug to fix immediately, not just a quality issue.
- If the AI cannot confidently work out what a request is actually asking for, it goes to a person for manual triage rather than being guessed at.
- During the first three months, we would check a sample of shortlists every week against what a staff member would have chosen by hand, and track how long each request actually takes, aiming to bring it down from about 7 hours a week to about 2.

Keeping the database current

- Startups get a link every few months to review and update their own listing. Changes go into a review queue rather than appearing live immediately.
- Any listing that hasn't been confirmed in 6 months is flagged and ranked lower until someone updates it.
- Whenever a staff member manually adds or removes a startup from a suggested shortlist, that correction is recorded, so repeated corrections can point to a rule that needs adjusting.

What ships in Week 1 vs. Month 3

	Week 1	Month 3
Where it runs	A lightweight prototype added to the current server, to get something working quickly	Its own separate app, running on the upgraded server
Database	The Excel file imported into a proper database table, a few hours of setup	The same data, kept up to date automatically
Matching	A staff member types the request into a simple search screen and gets a ranked list in seconds, instead of scrolling through Excel	Incoming emails are read and matched automatically, with nothing retyped
Email handling	Still read and handled by a person	Incoming emails are checked automatically; sending a shortlist is a single click
Quality check	None yet. This stage mainly establishes how long manual matching actually takes, as a baseline	Match quality and review time are tracked weekly against that baseline
Outcome	Removes most of the manual searching straight away, with very little build risk	Two apps running smoothly side by side, and very little manual work left per request

Week 1 is about proving the idea works quickly and measuring the real baseline. Month 3 is about giving it a proper, separate home once the rules have been tested against real requests.